

ExamGPT

AI-Powered Study Intelligence Platform

Multi-Agent RAG System for Exam Preparation

React

FastAPI

LangGraph

PostgreSQL

Neo4j

Redis

FAISS

Hugging Face

Complete Architecture • Full Stack Development • Cloud Deployment

AARON R

The Problem



Information Overload

Students drown in PDFs, notes, and study materials with no structured way to query them



No Context-Aware Search

Generic search engines can't understand exam-specific terminology or syllabus scope



No Source Attribution

AI chatbots give answers but never cite the source material — useless for exam prep



No Study Planning

Students lack personalized, AI-generated study plans based on their actual materials

Our Solution: ExamGPT



7-Node AI Pipeline

LangGraph state machine with planner, knowledge graph, retrieval, and evaluation



Hybrid Retrieval

BM25 + FAISS with Reciprocal Rank Fusion for best-of-both search



Source Citations

Every answer cites the exact PDF and page from your materials



Self-Correcting

Built-in evaluation loop retries poor answers up to 3 times

How It Works

Upload PDF → AI Extracts & Indexes → Ask Questions → 7-Agent Pipeline → Cited Answer + Score

Students upload course materials (PDFs), and ExamGPT builds a searchable knowledge base. Every question triggers a multi-agent pipeline that plans, retrieves, synthesizes, and evaluates the answer — with full source citations.

Technology Stack

Frontend

React 18 + Vite

SPA with HMR

React-Bootstrap

Responsive UI

Axios

HTTP client + Interceptors

Backend

FastAPI + Uvicorn

Async REST API

SQLAlchemy + Alembic

ORM + migrations

PyJWT + bcrypt

Auth & security

AI / ML

LangGraph

Multi-agent pipeline

Llama 3 8B (HF)

Text generation

MiniLM-L6-v2

384-dim embeddings

Data Stores

PostgreSQL (Supabase)

System of record

Neo4j AuraDB

Knowledge graph

Redis (Upstash)

Token cache

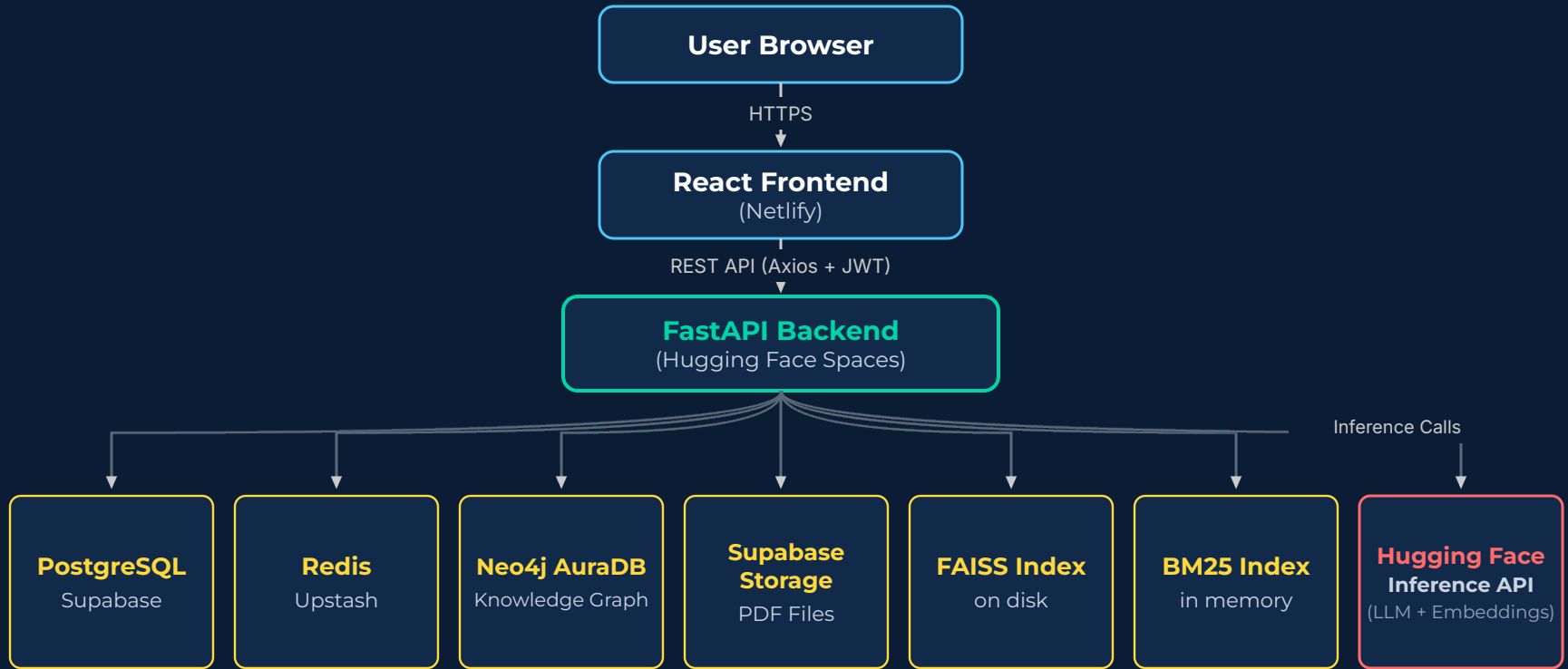
FAISS

Vector search

BM25

Keyword search

System Architecture



Request Flow — Asking a Question

01 User types a question in `ChatInterface.jsx`

02 `POST /api/v1/courses/id/query`

03 `query.py: submit_query()`

04 Verify course ownership in `PostgreSQL`

05 `run_examgpt_query()` invokes LangGraph pipeline

06 `Node Agent Pipeline` (see next diagram)

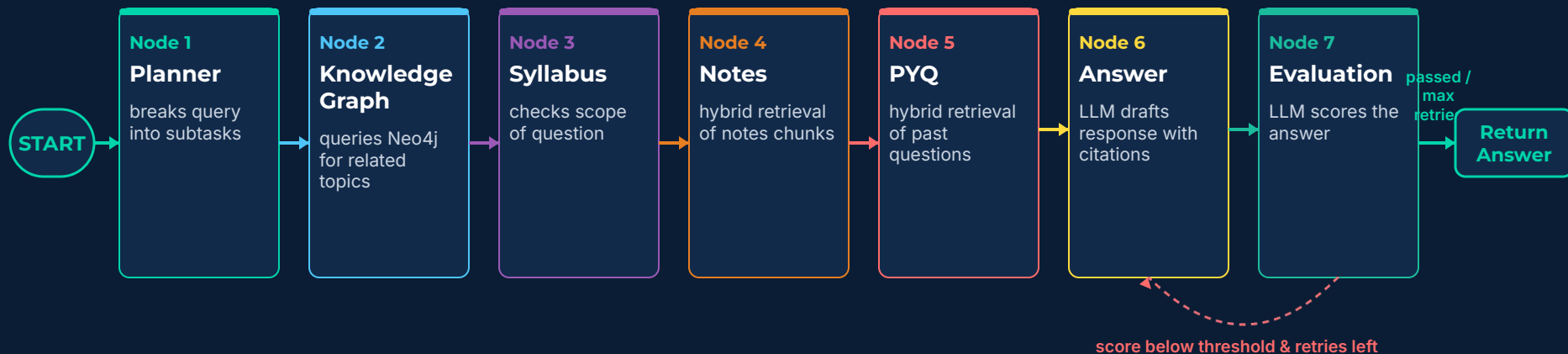
07 Save `ChatHistory` row to `PostgreSQL`

08 Save `PipelineRun` row (timing/observability)

09 Return `ChatResponse JSON` to frontend

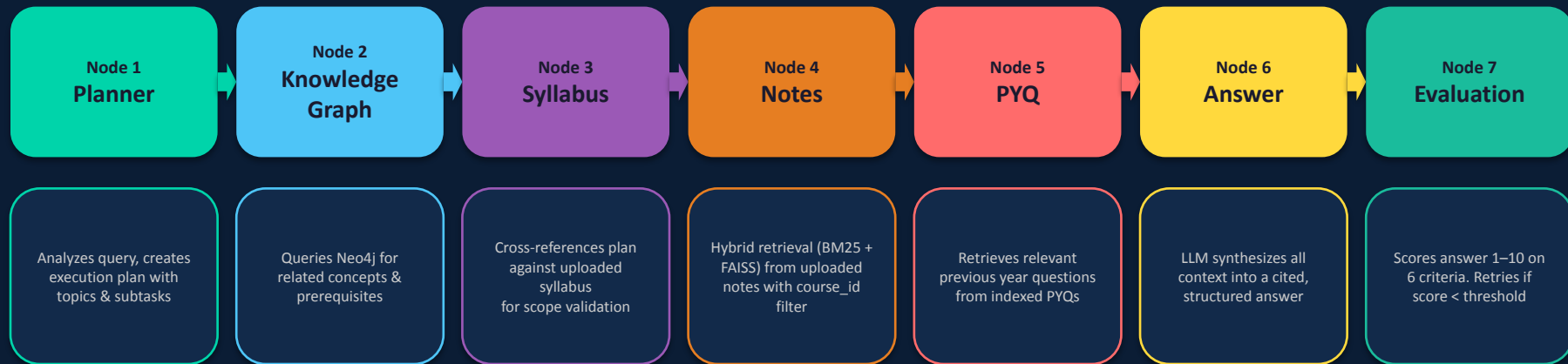
10 `ChatInterface.jsx` renders answer + citations

The 7-Node LangGraph Pipeline



Planner → Knowledge Graph → Syllabus → Notes → PYQ → Answer → Evaluation (with retry loop, max 3 attempts)

Pipeline Nodes — Deep Dive



Self-Correcting Retry Loop

- ▶ If evaluation score < threshold → Answer node re-generates with evaluation feedback
- ▶ Maximum 3 retry attempts to prevent infinite loops
- ▶ Evaluation checks: Correctness, Completeness, Syllabus Alignment, Citations, Clarity, Relevance
- ▶ Uses conditional edges in LangGraph — the `should_retry()` function controls the loop

Document Ingestion Pipeline

01. Upload

- User uploads PDF via `DocumentUpload.jsx`
- Triggers API endpoint `POST /documents`
- Handled by `upload_document()`

02. Store

- Validate file type and size limits
- Create Document row with status **processing**
- Save raw PDF bytes to Supabase Storage

03. Process

- **PDFLoader** text extraction
- `pypdf` reads and parses every page
- Chunker splits text into 1000-char overlapping chunks

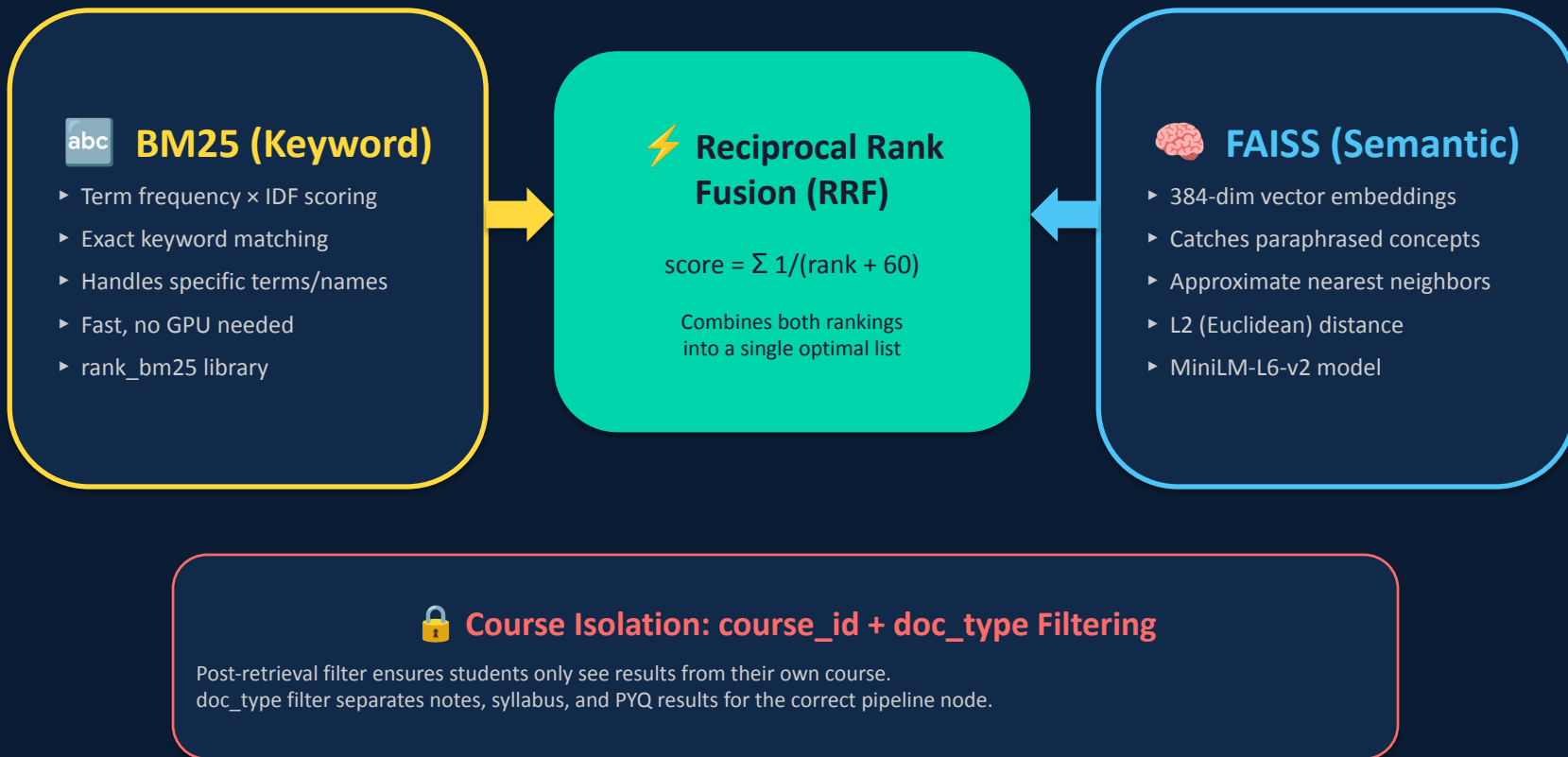
04. Index

- **hybrid_retriever** routes chunks
- Convert to 384-dim vectors & save to FAISS
- Rebuild BM25 index with keyword tokens

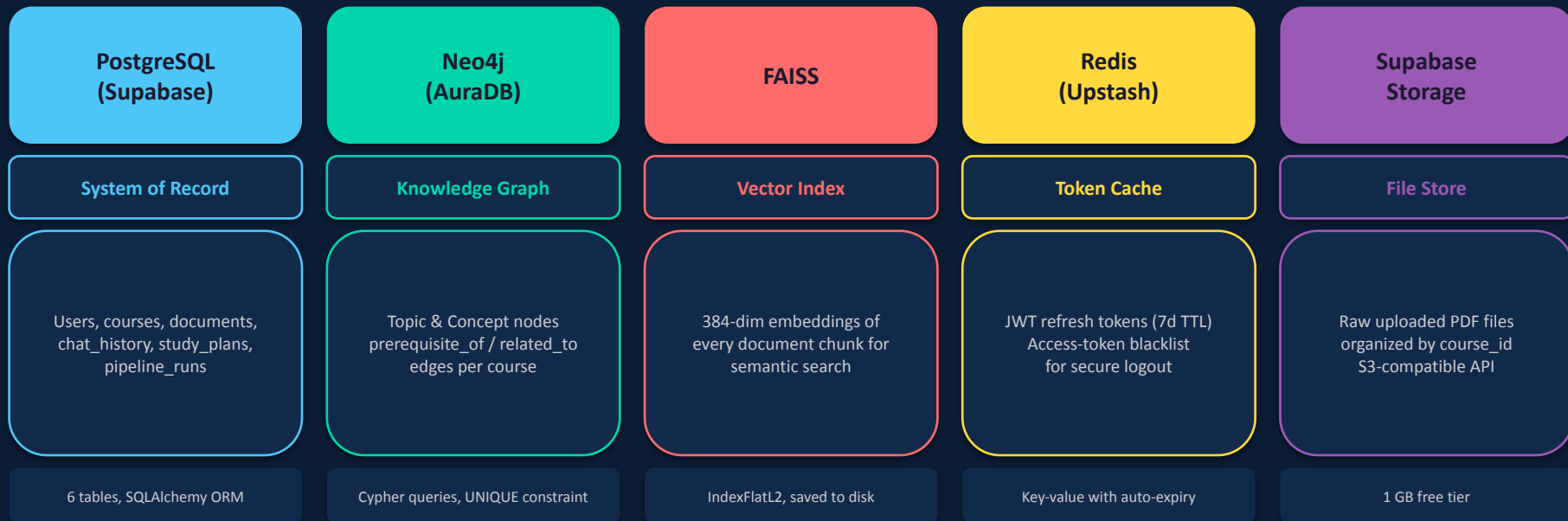
05. Ready

- Update Document row status to **ready**
- Respond to client with upload successful
- Pipeline completes successfully

Hybrid Retrieval System



Database Architecture



🔧 **Key Design: NullPool for SQLAlchemy** — HF Spaces kills idle connections. Each request creates a fresh DB connection.

Authentication & Security



JWT Authentication Flow

1. User sends username + password to `/api/v1/auth/login`
2. `bcrypt.checkpw()` verifies against stored hash
3. Server creates JWT with claims: `sub`, `exp`, `jti`, `type`
4. Access token (30 min) sent to client
5. Refresh token (7 days) stored in Redis with TTL
6. Client includes 'Authorization: Bearer <token>' in requests
7. `get_current_user()` decodes JWT on every protected endpoint
8. On logout: access token blacklisted, refresh token deleted



Security Features

► Password Hashing

`bcrypt` with automatic salting

► Stateless Auth

JWT — no server-side sessions

► Token Revocation

Redis blacklist for logged-out tokens

► CORS Protection

Whitelist of allowed frontend origins

► Owner Isolation

All data queries filtered by `user_id`

► Role-Based Access

`require_role()` factory for admin routes

► Env-Based Secrets

12-factor: no secrets in code

REST API Design



Authentication

POST /auth/register — Create account

POST /auth/login — Get JWT token

POST /auth/refresh — Renew token

POST /auth/logout — Revoke token

GET /auth/me — User profile



Courses

POST /courses/ — Create course

GET /courses/ — List courses

GET /courses/{id} — Get details

DEL /courses/{id} — Delete course



Documents

POST /courses/{id}/documents — Upload PDF

GET /courses/{id}/documents — List docs



AI / Chat

POST /courses/{id}/query — Ask question

GET /courses/{id}/chat-history — History

POST /courses/{id}/study-plan — Generate plan

GET /courses/{id}/study-plan — Get plan

Frontend Architecture

ENTRY & ROUTING

main.jsx

React Entry Point

Wraps app in AuthProvider + BrowserRouter

App.jsx

Router & Navbar

ProtectedRoute guards, Bootstrap navbar

APP VIEWS & PAGES

Login.jsx & Register.jsx

Auth Pages

Username/password form, email registration

Dashboard.jsx

Main Page

Course grid, create/delete modals

CourseDetail.jsx

Course Page

Tabs: Chat, Documents, StudyPlan

INNER COMPONENTS

ChatInterface

Real-time chat UI, citations, quality scores

DocumentUpload

PDF upload with file type selector

StudyPlanView

AI-generated dynamic study plan display

Key Patterns: AuthContext (global state) • Axios interceptors (auto JWT) • ProtectedRoute (auth guard) • React-Bootstrap (responsive UI)

Data Models — SQLAlchemy ORM

User

id, username (unique), email (unique), password_hash, name, role, created_at

Course

id, name, description, user_id (FK→User), created_at

Document

id, title, filename, doc_type, status, file_url, course_id (FK), user_id (FK)

ChatHistory

id, user_query, ai_response, citations_json, course_id, user_id

StudyPlan

id, plan_data (JSON), course_id, user_id, created_at, updated_at

PipelineRun

id, user_query, latency_ms, retry_count, success, course_id

Enums: UserRole (student | instructor | admin) • DocumentStatus (uploaded → processing → ready | failed) • DocumentType (syllabus | notes | pyq)

Key Features



PDF Upload & Processing

Upload course PDFs that are automatically extracted, chunked, embedded, and indexed



AI-Powered Chat

Ask questions about your materials and get answers with source citations



Quality Scoring

Every answer is evaluated on a 1-10 scale across 6 criteria



Study Plan Generator

AI creates personalized study plans based on your documents and timeline



Hybrid Search

BM25 + FAISS with RRF ensures both exact and semantic matches



Course Isolation

Each course has its own document index — no cross-contamination



Secure Auth

JWT tokens, bcrypt hashing, Redis blacklist, role-based access



Chat History

All conversations saved and retrievable with full citation records

Deployment Architecture

100% Free Cloud Tier Deployment

FREE

Frontend

Netlify

React build →
drag-and-drop deploy

FREE

Backend

HF Spaces

Docker container, git push
deploy

FREE

Database

Supabase

Managed PostgreSQL + S3
Storage

FREE

Graph DB

Neo4j AuraDB

Knowledge graph, 50k
nodes free

FREE

Cache

Upstash Redis

Token store, 10k cmds/day

FREE

AI/LLM

HF Inference

Llama 3 8B + MiniLM
embeddings



Total Monthly Cost: \$0.00

Key Design Decisions

Why LangGraph?

Typed, compiled state machine vs fragile function chains.
Conditional edges enable the retry loop naturally.

Why Hybrid Retrieval?

FAISS catches paraphrased concepts. BM25 catches exact terms.
RRF combines both for optimal results.

Why NullPool?

HF Spaces kills idle connections. NullPool creates a
fresh DB connection per request — slower but 100% reliable.

Why Supabase?

Free managed PostgreSQL + S3 storage + dashboard.
Standard PostgreSQL under the hood — no vendor lock-in.

Why `course_id` filtering?

Prevents cross-course data leakage. Without it, Course A
questions would retrieve Course B documents.

Why 12-factor secrets?

Environment variables only — no secrets in code or git.
Different environments just change the env vars.

Future Roadmap

Phase 1 — Near Term

- ▶ Async document processing with Celery
- ▶ Document deletion from vector indexes
- ▶ Streaming responses (SSE) for real-time chat
- ▶ Rate limiting via Redis

Phase 2 — Medium Term

- ▶ Multi-modal document support (images, diagrams)
- ▶ Quiz generation from uploaded materials
- ▶ Collaborative courses (shared study groups)
- ▶ Progress tracking and analytics dashboard

Phase 3 — Long Term

- ▶ Fine-tuned domain-specific LLM
- ▶ Voice-based Q&A interface
- ▶ Mobile app (React Native)
- ▶ Instructor analytics and content insights

Thank You!

ExamGPT — AI-Powered Study Intelligence Platform

35+

Source Files

7

AI Agents

17

API Endpoints

5

Data Stores

\$0

Monthly Cost

Questions & Discussion